# The C++ Standard Template Library

Containers hold data, algorithms work on it, iterators connect them. Container choice, operation costs and iterator invalidation are the parts that matter most.

## The big picture




Algorithms never touch containers, only iterator ranges.

One sort works on any container that offers the right iterators.

| PART | WHAT IT IS | EXAMPLES |
|---|---|---|
| **Containers** | classes that own and organise a collection of elements | `vector`, `map`, `set`, `deque` |
| **Iterators** | generalised pointers that walk a container | `v.begin()`, `m.find(k)` |
| **Algorithms** | generic functions that operate on iterator *ranges* | `sort`, `find`, `accumulate` |

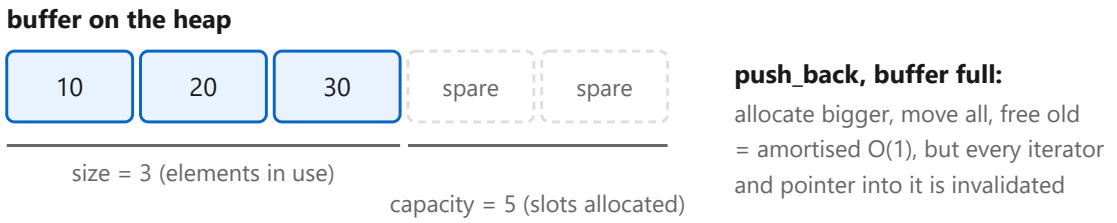
Algorithms see only `[first, last)`, so one `std::sort` serves a vector, array or deque. That separation is the design.

```
#include <vector>       // std::vector
#include <map>          // std::map, std::multimap
#include <unordered_map>// std::unordered_map
#include <set>          // std::set
#include <algorithm>    // std::sort, std::find, std::transform ...
#include <numeric>      // std::accumulate, std::iota
#include <string>       // std::string
```

**DEFAULT CHOICE** `std::vector`. Contiguous memory is so cache-friendly it often beats "better" containers.

---

## `vector`: the workhorse




reserve() grows capacity only; resize() changes size.

A `vector` is one contiguous heap array with a **size** and a **capacity**. When full, `push_back` allocates a bigger buffer, moves everything and frees the old one: occasional O(n), **amortised O(1)**.

```
std::vector<int> v;
v.reserve(100);          // capacity 100, size still 0: no reallocation for
the next 100 push_backs
v.push_back(1);          // copy/move an existing object in
v.emplace_back(2);       // construct the element in place: no temporary
v.resize(10);            // size becomes 10; new elements are value-
initialised (0)

v[3];                    // no bounds check: out-of-range is undefined
behavior
v.at(3);                 // bounds-checked: throws std::out_of_range
v.front(); v.back(); v.data();   // data() is the raw int* for C APIs
v.size(); v.capacity(); v.empty();
```

**RESERVE VS RESIZE** `reserve(n)` only allocates; `resize(n)` creates elements. `v[5] = x` after only `reserve(10)` is UB.

**BRACES VS PARENTHESES**

```
std::vector<int> a(3, 7);   // parentheses: size 3, every element 7  -
> {7, 7, 7}
std::vector<int> b{3, 7};   // braces: the initializer_list constructor
wins  →  {3, 7}
```

With braces, a constructor taking `std::initializer_list` is preferred whenever it can match.

---

## Iterators

An iterator acts like a pointer: `*it`, `++it`, `it == end`. `end()` is **one past the last**; ranges are half-open `[begin, end)`.

```
std::vector<int> v{1, 2, 3};
for (auto it = v.begin(); it ≠ v.end(); ++it) std::cout << *it;   // use ≠
not <, it works for every container
for (int x : v) std::cout << x;           // range-for is exactly this loop
in disguise
for (auto it = v.rbegin(); it ≠ v.rend(); ++it) { }   // reverse iterators
for (auto it = v.cbegin(); it ≠ v.cend(); ++it) { }   // cbegin/cend: read-
only, *it cannot be modified
auto it2 = std::next(v.begin(), 2);       // advance without assuming random
access
```

| CATEGORY | CAN DO | CONTAINERS |
|---|---|---|
| Forward | `++`, read (multi-pass) | `forward_list`, unordered containers |
| Bidirectional | also `--` | `list`, `map`, `set` |
| Random access | also `it + n`, `it[i]`, `it1 - it2`, `<` | `vector`, `deque`, `array`, `string` |

So `std::sort` needs random access and fails on `std::list` (use `list.sort()`).

### Iterator adaptors: filling a container

An algorithm like `std::copy` **assigns** through its output iterator, so the destination must already be big enough. An **inserter** turns each assignment into an insertion instead:

| ADAPTOR | EACH ASSIGNMENT DOES |
| --- | --- |
| `std::back_inserter(c)` | `c.push_back(x)` |
| `std::front_inserter(c)` | `c.push_front(x)` (so the order reverses; needs `deque` / `list`) |
| `std::inserter(c, pos)` | `c.insert(pos, x)` (works for `set` and `map`) |

```
std::vector<int> src{3, 1, 2}, out;
std::copy(src.begin(), src.end(), std::back_inserter(out));      // out is
{3, 1, 2}: it grew
// std::copy(src.begin(), src.end(), out.begin());               // BUG if
out is empty: writes past the end

std::set<int> sorted;
std::copy(src.begin(), src.end(), std::inserter(sorted, sorted.begin()));
```

`std::reverse_iterator` (what `rbegin()` returns) and `std::move_iterator` (moves instead of copying) are the other two adaptors worth knowing.

### Iterator invalidation

Changing a container can **invalidate** iterators, pointers and references; using one is UB.

| CONTAINER | OPERATION | WHAT IS INVALIDATED |
|---|---|---|
| `vector` | `push_back`, `insert` | **all** if it reallocated; otherwise those at or after the insertion point |
| `vector` | `erase` | the erased element and everything after it |
| `deque` | insert/erase in the middle | everything; push at either end invalidates iterators but not references |
| `list`, `map`, `set` | insert | nothing |
| `list`, `map`, `set` | erase | only the erased element |
| `unordered_*` | insert that triggers a rehash | all iterators (references stay valid) |

▼ SPOT THE BUG **Erasing inside a loop**

```cpp
std::vector<int> v{1, 2, 3, 4};
for (auto it = v.begin(); it ≠ v.end(); ++it)
    if (*it % 2 == 0) v.erase(it);          // BUG
```

`erase` invalidates `it`, and the loop then does `++it` on a dead iterator. The correct idiom uses the iterator `erase` returns:

```cpp
for (auto it = v.begin(); it ≠ v.end(); ) {
    if (*it % 2 == 0) it = v.erase(it);     // erase returns the next
valid iterator
    else ++it;
}
// or, in C++20:
std::erase_if(v, [](int x) { return x % 2 == 0; });
```

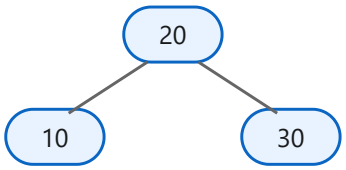
## Other sequence containers

| CONTAINER | STORAGE | BEST AT | WATCH OUT |
|---|---|---|---|
| `std::array<T,N>` | fixed size, inline (stack) | size known at compile time; zero overhead | cannot grow |
| `std::vector<T>` | one contiguous heap block | almost everything; fast iteration and indexing | insert/erase in the middle is O(n) |
| `std::deque<T>` | chunks of arrays | fast push/pop at **both** ends, still indexable | not contiguous |
| `std::list<T>` | doubly linked nodes | O(1) insert/erase anywhere *if you hold an iterator*; iterators stay valid | no indexing, a heap allocation per element, poor cache behaviour |
| `std::forward_list<T>` | singly linked nodes | minimal memory list | forward iteration only |

| OPERATION | VECTOR | DEQUE | LIST |
|---|---|---|---|
| index `[i]` | O(1) | O(1) | not available |
| `push_back` | amortised O(1) | O(1) | O(1) |
| `push_front` | O(n) | O(1) | O(1) |
| insert / erase in the middle | O(n) | O(n) | O(1) at a known position |
| find a value | O(n) | O(n) | O(n) |

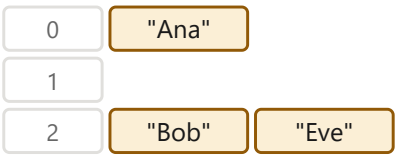
# Associative and unordered containers

**map / set: balanced tree, sorted, O(log n)**




in-order walk = sorted order

**unordered_map: hash table, average O(1)**




hash(key) picks a bucket; collisions chain (Bob, Eve)

Need order or range queries: map. Plain lookup: unordered_map.

| | `MAP` / `SET` | `UNORDERED_MAP` / `UNORDERED_SET` |
|---|---|---|
| Implementation | balanced binary tree (usually red-black) | hash table |
| Order of iteration | **sorted** by key | unspecified |
| find / insert / erase | O(log n) | average O(1), worst case O(n) |
| Key requirement | `operator<` (a strict weak ordering) | a hash function and `operator==` |
| Range queries (`lower_bound`) | yes | no |

```
std::map<std::string, int> age;
age["Ana"] = 30;                          // operator[] INSERTS a default (0)
if the key is missing
age.insert({"Bob", 25});                  // does nothing if "Bob" already
exists
age.insert_or_assign("Bob", 26);          // C++17: insert or overwrite

if (auto it = age.find("Ana"); it ≠ age.end())   // find never inserts
    std::cout << it→second;                       // it→first is the key,
it→second the value

for (const auto& [name, years] : age)     // structured bindings; sorted by
name
    std::cout << name << ' ' << years << '\n';
```

```cpp
std::unordered_set<int> seen;
if (!seen.insert(5).second) { /* second == false means it was already there */ }
```

## `try_emplace`, `multimap` and `multiset`

| CALL | IF THE KEY ALREADY EXISTS |
|---|---|
| `m[k] = v` | overwrites (and inserts a default first if missing) |
| `m.insert({k, v})` / `m.emplace(k, v)` | does nothing, but **may already have moved from or built the value** |
| `m.try_emplace(k, args ... )` (C++17) | does nothing and **guarantees the arguments are not touched**: compiled, a moved-in 40-character string was intact afterwards |
| `m.insert_or_assign(k, v)` (C++17) | overwrites |

```cpp
std::map<int, std::string> a{{1, "old"}};
std::string payload(40, 'p');
a.try_emplace(1, std::move(payload));     // key 1 exists: nothing happens, payload still holds 40 chars
a.try_emplace(2, std::move(payload));     // new key: the string is moved into the map

std::multimap<std::string, int> mm{{"fruit", 1}, {"fruit", 2}, {"veg", 9}};
// duplicate keys allowed
auto [lo, hi] = mm.equal_range("fruit");   // the range of all entries with that key
mm.count("fruit");                         // 2: there is no operator[] because a key may have many values
```

- `multimap` and `multiset` allow duplicate keys and keep equal keys in insertion order. There is no `operator[]`.
- `try_emplace` is the right call for "insert if absent" when building the value is expensive or the argument is move-only.

### Container adaptors

They wrap another container and restrict the interface: `std::stack` (LIFO: `push/pop/top`), `std::queue` (FIFO: `push/pop/front`) and `std::priority_queue` (a heap: `top()` is the **largest** element by default).

```
std::priority_queue<int> maxHeap;                                  // largest first
std::priority_queue<int, std::vector<int>, std::greater<int>> minHeap;   // smallest first
```

## Which container should I use?

| I NEED TO... | USE | WHY |
|---|---|---|
| iterate over items, access by index, mostly append at the end | `std::vector` | Contiguous memory: fastest iteration and indexing, amortised O(1) `push_back`. This is the default choice. |
| hold a fixed number of items known at compile time | `std::array` | Lives inline (no heap), knows its size, zero overhead over a C array. |
| push and pop at both ends (a deque of work) | `std::deque` | O(1) push/pop at front and back while still allowing indexing. |
| insert/erase often in the middle while holding iterators that must stay valid | `std::list` | O(1) splice/insert/erase at a known position and stable iterators. Costs: a heap allocation per node and poor cache locality. Measure before choosing it. |
| look up values by key; order does not matter | `std::unordered_map` | Hash table: average O(1) find/insert/erase. Needs a hash and `operator==`. |
| look up by key AND iterate in sorted order, or find the nearest key | `std::map` | Balanced tree: O(log n), sorted iteration, `lower_bound` / `upper_bound` for range queries. |
| check membership / remove duplicates (no values attached) | `std::unordered_set` | Hash set: average O(1) contains. Use `std::set` if you also need the elements sorted. |
| count how often each word occurs | `std::unordered_map<std::string, int>` | `counts[word]++` inserts a zero on first sight and increments in one expression. |
| last-in first-out (undo, DFS, matching brackets) | `std::stack` | Adaptor over deque/vector exposing only push, pop and top. |
| first-in first-out (task queue, BFS) | `std::queue` | Adaptor over deque exposing push, pop, front and back. |
| always process the largest (or smallest) item next | `std::priority_queue` | A binary heap: O(log n) push/pop, O(1) top. Largest first by default; pass `std::greater` for smallest first. |

KEY POINT "vector vs list?" Answer with *memory*: contiguous means cache-friendly, so vector wins even for middle inserts at modest sizes. List wins for stable iterators and cheap splicing.

---

## Custom keys: comparators and hashes

A `map` / `set` needs an **ordering**; an `unordered_map` / `unordered_set` needs a **hash and equality**. For your own key types, you provide them.

```
struct NoCase {                                          // an ordering that
ignores case
    bool operator()(const std::string& a, const std::string& b) const {
        return std::lexicographical_compare(a.begin(), a.end(), b.begin(),
b.end(),
            [](unsigned char x, unsigned char y) { return std::tolower(x) <
std::tolower(y); });
    }
};
std::map<std::string, int, NoCase> m;                    // the comparator is
part of the TYPE
m["Alpha"] = 1; m["ALPHA"] = 2;                          // the same key:
size is 1 (compiled: 2 keys with "beta")

struct Product { std::string shop; int id; bool operator==(const Product&)
const = default; };
struct ProductHash {
    std::size_t operator()(const Product& p) const {
        std::size_t h = std::hash<std::string>{}(p.shop);
        h ^= std::hash<int>{}(p.id) + 0x9e3779b97f4a7c15ULL + (h << 6) + (h
>> 2);   // combine the member hashes
        return h;
    }
};
std::unordered_map<Product, double, ProductHash> price;
price[{"north", 7}] = 101.5;
```

| CONTAINER | KEY MUST PROVIDE | RULE |
| --- | --- | --- |
| `map`, `set` | a **strict weak ordering** (`operator<` or a comparator) | irreflexive and transitive; equivalence means "neither is less". `≤` is **not** valid: undefined behaviour |
| `unordered_map`, `unordered_set` | a hash and `operator==` | equal keys **must** have equal hashes; a poor hash makes every lookup O(n) |

- **Transparent comparators:** `std::map<std::string, int, std::less<>>` lets `find` take a `string_view` or `const char*` without building a temporary `std::string` (compiled: it found `"apple"` from a `string_view`). C++20 adds the same for unordered containers.
- **Never change a key** while it is in the container: its position or bucket would be wrong. Keys are `const` for that reason.
- **Hash quality:** combine *all* fields; avoid returning just one field. `std::hash` for `int` is the identity in libstdc++, which is fine.
- A `rehash` (growth beyond `max_load_factor`) invalidates iterators; `reserve(n)` up front avoids it, and avoids the allocations on a hot path.
- Ordered or hashed? Ordered when you need sorted iteration or range queries (`lower_bound`); hashed for plain lookup. For a few dozen keys a sorted `vector` and binary search often beats both.

KEY POINT "How do you use your own type as a key in `unordered_map`?" Provide `operator==` and a hash functor (or specialise `std::hash`), combining all members, and make sure equal objects hash equally.

## Algorithms

Over a hundred generic functions in `<algorithm>` / `<numeric>`. Prefer them to loops: named, tested, often faster. Most take a lambda.

```
std::sort(v.begin(), v.end(), [](const P& a, const P& b) { return a.age < b.age; });   // custom order
auto it = std::find_if(v.begin(), v.end(), [](int x) { return x > 6; });
int n  = std::count_if(v.begin(), v.end(), [](int x) { return x % 2 == 0; });
bool any = std::any_of(v.begin(), v.end(), [](int x) { return x < 0; });
int sum  = std::accumulate(v.begin(), v.end(), 0);          // <numeric>
std::transform(v.begin(), v.end(), v.begin(), [](int x) { return x * x; });
auto pos = std::lower_bound(sorted.begin(), sorted.end(), 5);   // binary search: range MUST be sorted
```

> **STRICT WEAK ORDERING** Use `<`, never `≤`: a comparator true for equal elements can crash `sort` or corrupt a `set`. `sort` is not stable; use `stable_sort`.

## Ranges and views (C++20)

```
#include <ranges>

std::ranges::sort(v);                                       // pass the container, not begin() and end()
std::ranges::sort(v, std::greater<>{});                     // descending (compiled: first element 6)

auto evenSquares = v | std::views::filter([](int x) { return x % 2 == 0; })
                     | std::views::transform([](int x) { return x * x; });
// a LAZY view: nothing is computed yet
for (int x : evenSquares) use(x);                           // computed on demand, no temporary vector (compiled: sum 56)
```

- A **range** is anything with `begin()` and `end()`. The `std::ranges::` algorithms take the whole range and support **projections**: `std::ranges::sort(people, {}, &Person::age)`.

- A **view** is a lightweight, non-owning, lazy window (`filter`, `transform`, `take`, `drop`, `reverse`, `iota`). Pipe them with `|`.
- A view does not own its data: if the container dies or reallocates, the view dangles, like `string_view`.
- Ranges make pipelines readable; you do not need to know every view.

---

## Common pitfalls

`pair`, `tuple`, `optional` and `variant` are covered in Vocabulary Types.

| PITFALL | FIX |
| --- | --- |
| Using an iterator/pointer/reference after a `push_back` reallocated the vector | `reserve` up front, or re-fetch the iterator, or index instead |
| `map[key]` silently inserts | use `find` / `contains` to test; `at()` to read-or-throw |
| `v.size() - 1` on an empty vector (`size()` is unsigned, so it wraps to a huge number) | check `empty()` first or write `i + 1 < v.size()` |
| Passing a big container by value | pass `const std::vector<T>&` |
| `std::vector<bool>` is a packed-bit special case with proxy references | use `std::vector<char>` or `std::bitset` |
| `auto x = v[0]` copies the element | `const auto& x = v[0]` |
| `std::find` on a `map` is O(n) | use the member `m.find(key)`, O(log n) |

## Random numbers: `<random>`

```
std::mt19937 gen(12345);                          // an ENGINE: produces raw
random bits (here seeded for repeatability)
std::uniform_int_distribution<int> dice(1, 6);    // a DISTRIBUTION: shapes
the bits into the numbers you want
int roll = dice(gen);

std::random_device rd;                            // a non-deterministic seed
source (hardware or OS)
std::mt19937 gen2(rd());
std::normal_distribution<double> noise(0.0, 1.0);
```

- **Engine plus distribution**: never use `rand() % n` (poor quality, biased, global state). Same seed gives the **same sequence** (compiled: true), which is what you want in tests and simulations.
- Compiled: 6000 rolls of the dice gave every face between 850 and 1150 times; 10,000 normal samples had a mean near 0.
- Keep one engine per thread; engines are not thread-safe.

## Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

| SECTION | BOOK | PART OF THE BOOK | PDF PAGES |
|---|---|---|---|
| **vector: the workhorse** | C++ Primer | 3.3 Library vector type | 125 to 134 |
| | C++ Primer | 9.3 Sequential container operations | 370 to 383 |
| | C++ Primer | 9.4 How a vector grows | 384 to 388 |
| | Programming: Principles and Practice | 3.5 to 3.6 Functions; vector introduced | 81 to 89 |
| | Programming: Principles and Practice | 17.6 to 17.9 Other operations, remaining Vector problems, changing size | 513 to 523 |
| | Effective Modern C++ | Item 42, Consider emplacement instead of insertion | 310 to 320 |
| | C++ Move Semantics | Chapter 15 Move semantics in standard library types | 229 to 250 |
| **Iterators** | C++ Primer | 3.4 Introducing iterators | 135 to 141 |
| | C++ Primer | 10.4 Revisiting iterators | 430 to 438 |
| | C++ Primer | 10.5 Structure of generic algorithms | 439 to 443 |
| | Programming: Principles and Practice | 19.1 to 19.3, 19.6 Containers, sequences and iterators, linked lists, vector/list/string | 559 to 572 and 585 to 586 |
| **Other sequence containers** | C++ Primer | 9.1 to 9.2 Overview and the container library | 355 to 369 |
| | C++ Primer | 9.6 Container adaptors | 397 to 402 |
| | C++ Primer | 10.6 Container-specific algorithms | 444 to 447 |
| **Associative and unordered containers** | C++ Primer | 11.1 to 11.2 Using an associative container, overview | 449 to 456 |
| | C++ Primer | 11.3 Operations on associative containers | 457 to 471 |
| | Programming: Principles and Practice | 20.1 to 20.6 map, unordered_map, set, container overview | 591 to 609 |
| **Which container should I use?** | Programming: Principles and Practice | 20.1 to 20.6 map, unordered_map, set, container overview | 591 to 609 |

| SECTION | BOOK | PART OF THE BOOK | PDF PAGES |
|---|---|---|---|
| **Custom keys: comparators and hashes** | C++ Primer | 11.3 Operations on associative containers | 457 to 471 |
| | C++ Primer | 11.4 The unordered containers | 472 to 477 |
| **Algorithms** | C++ Primer | 10.1 to 10.2 Overview and a first look | 405 to 413 |
| | Programming: Principles and Practice | 21.1 to 21.2, 21.5 Standard algorithms, function objects, sorting and searching | 617 to 626 and 633 to 635 |
| | C++ Move Semantics | Chapter 14 Moving algorithms and iterators | 219 to 228 |
| **Common pitfalls** | C++ Primer | 9.3 Sequential container operations | 370 to 383 |
| **Random numbers** | C++ Primer | 17.4 Random numbers | 774 to 780 |